

Технічне завдання (ТЗ): веб-платформа створення та керування чат-ботами

1. Система користувачів

Платформа повинна підтримувати реєстрацію та аутентифікацію користувачів. Реєстрація/логіні здійснюються через email/пароль; додатково можна надати опціональну можливість входу через соціальні мережі (Facebook, Google тощо). Для цього доцільно використати стандартну систему автентифікації Django разом із пакетом **django-allauth** для соціальної аутентифікації (Allauth – інтегрований набір додатків для реєстрації та соціального логіну ¹). Користувачі поділені на ролі: звичайний користувач та адміністратор. Реалізувати це можна через групи та дозволи Django – гнучку систему RBAC (role-based access control) ². Кожен користувач має профіль, у якому зберігається персональна інформація та обраний тарифний план.

- **Реєстрація та логін:** email/пароль (через Django Auth) із опцією соцмереж (django-allauth) ¹.
- **Ролі користувачів:** «User» і «Admin», реалізовані через Django Groups/Permissions ².
- **Профіль користувача:** зберігає особисті дані та обраний тарифний план.

2. Створення ботів

Користувачі можуть створювати один або кілька ботів. При створенні нового бота користувач вибирає платформу – Telegram, Viber або WhatsApp. Для кожного бота необхідно налаштувати підключення до відповідного API: зберігати і використовувати ключі/токени і налаштовувати вебхуки для отримання повідомлень.

- **Множинність:** один користувач може мати багато ботів; користувач обирає платформу для кожного бота.
- **Telegram:** підключення через Telegram Bot API. Потрібно отримати токен бота від BotFather і використовувати його для запитів до API ³.
- **Viber:** підключення через Viber REST API. Для аутентифікації використовується унікальний токен (X-Viber-Auth-Token) вашого бота ⁴.
- **WhatsApp:** підключення через офіційний WhatsApp Business API або провайдер (наприклад, через Twilio). Наприклад, можна використовувати **Twilio WhatsApp API** для пересилки повідомлень і отримання відгуків від користувачів ⁵.
- **Webhook:** для кожної платформи налаштувати HTTPS-ендпоінт (webhook), куди API надсилатиме події та повідомлення від користувачів (сервер повинен обробляти такі запити).

3. Редактор ботів

Інтерфейс редагування бота надавати у вигляді візуального конструктора логіки діалогу. Користувач має можливість створювати *чат-флоу* (дерево рішень) з умовами (if/then), тригерами та діями. Наприклад, бот може задавати запитання з кнопками, аналізувати введені слова або натискання кнопок і переходити до наступного кроку. Потрібні функції:

- Авто-відповіді (попередньо задані відповіді на стандартні питання).
- Умови та правила переходу (if-клаузи, регулярні вирази тощо).
- Кнопкові меню (Quick Reply, Inline Buttons), відправка текстових і медіа-повідомлень (зображень, відео тощо).
- **Інтеграції:** опційно зв'язок з Google Sheets (можливість імпорту/експорту даних до таблиць) або власні Webhook-дії (наприклад, виклик зовнішнього API під час діалогу).

Напрямок роботи: розробити «drag-and-drop» або подібний графічний інтерфейс для налаштування блоків діалогу. Зберігати дерева рішень у базі даних, щоб при отриманні повідомлення від бота виконувати відповідний сценарій. (Сторонніх джерел, специфічних для цього функціоналу, достатньо поширено в документації по чат-ботам і не наведено тут.)

4. Функції ботів

У роботі ботів реалізувати такі можливості:

- **Збір даних:** бот має можливість накопичувати інформацію від користувачів (наприклад, імена, телефони, email) на основі введених відповідей або форм.
- **Інтеграція з формами:** можна зв'язати бота з веб-формами (наприклад, якщо є web-форма, дані з неї можуть передаватися у бота і навпаки).
- **Логування повідомлень:** зберігання історії повідомлень і взаємодій для кожного бота (щоб можна було переглянути логи діалогів).

Ця функціональність забезпечує, що бот може зберігати корисні дані та аналітику спілкування, що важливо для статистики і подальшого аналізу.

5. Панель керування

Веб-інтерфейс адміністратора-платформи (після входу користувач бачить своїх ботів) повинен включати:

- **Список ботів:** сторінка з усіма створеними ботами користувача (назва, платформа, статус).
- **Статистика по ботах:** для кожного бота відображати ключові метрики – кількість зареєстрованих користувачів бота, кількість отриманих і відправлених повідомлень, число взаємодій (кнопкових кліків, завершених сценаріїв тощо).
- **Управління ботом:** кнопки/опції для редагування налаштувань бота, видалення чи тимчасового відключення (деактивації) бота.
- **Моніторинг:** повідомлення про помилки або статус вебхуків (опціонально) – наприклад, якщо бот не зміг доставити повідомлення.

Ця панель дозволяє користувачу контролювати роботу ботів і аналізувати їхню ефективність.

6. Платна підписка

Реалізувати систему тарифних планів для користувачів. Кожний тарифний план може накладати обмеження на кількість ботів, кількість повідомлень на місяць, доступні функції (наприклад, базовий, бізнес, корпоративний). Необхідно:

- **Тарифні плани:** зберігати налаштування кожного плану (кількість ботів, дозволені функції, ліміти по повідомленням тощо). Користувач при реєстрації чи оновленні акаунту обирає план.
- **Оплата:** інтегрувати платіжну систему (наприклад, Stripe). Можна використати бібліотеку **dj-stripe**, яка спрощує роботу зі Stripe у Django (підтримує створення підписок, вебхуків для оновлення статусу оплати тощо) ⁶.

- **Платіжні вебхуки:** обробка webhook-викликів від платіжної системи (надання доступу після успішної оплати, автоматичне списання підписок тощо).

Головне – забезпечити безперебійну роботу підписки: створення пробного періоду, списання коштів за підписку, можливість оновити або скасувати план.

7. Адмін-панель

Потрібна окрема адмін-панель для адміністраторів платформи (звичайного користувача з роллю Admin). Функції:

- **Перегляд користувачів і ботів:** список усіх зареєстрованих користувачів та їхніх ботів. Можливість фільтрувати, шукати та переглядати деталі.
- **Статистика загалом:** узагальнена статистика по платформі – кількість користувачів, ботів, загальна активність (для моніторингу зростання).
- **Управління тарифами:** можливість створювати і змінювати тарифні плани та їхні параметри.
- **Блокування користувачів:** бан/деактивація акаунтів порушників.
- **Перегляд логів:** централізовані логи помилок сервера, системних повідомлень, API-запитів.

Цю панель можна реалізувати на базі стандартного **Django Admin** (розширеного потрібними моделями) або через окремий інтерфейс за API.

8. Технічні вимоги

- **Бекенд:** Django з Django REST Framework (DRF) для побудови REST-API. DRF забезпечує валідацію даних і гнучкі серіалізатори для всіх моделей.
- **Фронтенд:** Vite + React. Vite – сучасний бандлер, що дозволяє швидкий hot-reload і швидко компіляцію (швидше за Webpack) ⁷. React – компонентний фреймворк для побудови динамічного UI (можна використовувати JSX-компоненти, бібліотеки UI-компонентів).
- **База даних:** PostgreSQL – надійна SQL БД, добре масштабується з Django ORM (як показано в документації DRF/Twilio, PostgreSQL – звичайний вибір для подібних застосунків ⁵).
- **Асинхронні завдання:** Celery + Redis. Використовувати **Celery** для фонові обробки трудомістких чи періодичних завдань (наприклад, відправка повідомлень ботам або робота з вебхуками). Celery – це система розподіленого виконання задач (task queue), що обробляє завдання асинхронно, не блокуючи основний потік додатку ⁸. Для Celery слід налаштувати повідомлювач (broker) – наприклад, Redis або RabbitMQ ⁹ (Redis є поширеним вибором).
- **Масштабована архітектура:** застосувати контейнеризацію (Docker). Контейнеризація упакує додаток із усіма залежностями в портативний контейнер ¹⁰, що забезпечує консистентність між середовищами розробки, тестування та продакшену. Розгортання рекомендується автоматизувати через оркестрацію (Kubernetes), щоб легко масштабувати сервіси на хмарній інфраструктурі (AWS, GCP, Azure тощо) ¹¹.

Додатково: налаштувати CI/CD для автоматичних деплоїв, використовувати Nginx або подібний проксі для обслуговування HTTPS і статичних файлів, слідкувати за версіями залежностей.

9. Безпека

- **Аутентифікація та авторизація:** застосувати захищені методи автентифікації (JWT, OAuth2 тощо) і перевіряти ролі користувачів на кожному REST-запиті. Django REST Framework має механізми дозволів (permissions) та throttling для контролю доступу.

- **Захист API:** використовувати HTTPS на всіх ендпоінтах – шифрування TLS обов'язкове ¹². Секретні ключі й токени зберігати у файлі середовища (.env) чи у сховищі секретів, а не у відкритому коді – це захистить їх від витоку ¹³.
- **Rate limiting:** обмежувати частоту запитів до загальнодоступних ендпоінтів. Наприклад, DRF пропонує `UserRateThrottle` / `AnonRateThrottle` – можна налаштувати ліміти, скажімо, 1000 запитів на день для аутентифікованого користувача і 100/день для аноніма ¹⁴, або встановити десятки запитів на хвилину ¹⁵. Це запобігатиме перевантаженню API (також можна обмежувати окремі чутливі маршрути).
- **Шифрування даних:** усі конфіденційні дані (наприклад, API-ключі, паролі) – шифрувати у базі або зберігати хешованими. Створювати `HttpOnly+Secure` куки для токенів сесії, якщо застосовується.
- **Логи та моніторинг:** вести аудит-логи для фіксації помилок аутентифікації, несанкціонованих доступів, змін тарифних планів тощо. Регулярно перевіряти безпеку сервера, оновлювати залежності.

Загалом, слід дотримуватись рекомендацій безпеки Django та галузевих практик (OWASP). Наприклад, використовувати `django-rest-framework-simplejwt` чи `django-rest-knox` для безпечних токенів; поширені поради: “**HTTPS повсюди**” і “короткоживучі токени з ротацією” ¹².

10. Інтерфейс

- **Адаптивний дизайн:** інтерфейс сайту повинен коректно відображатися на десктопах, планшетах і мобільних пристроях. Мобільна версія повинна бути зручною для користувачів (responsive layout).
- **Мультимовність:** підтримка інтернаціоналізації (i18n). Інтерфейс – принаймні українською, англійською та російською мовами. Django має вбудовану систему перекладів для шаблонів і рядків інтерфейсу ¹⁶, а на фронтенді можна використати, наприклад, бібліотеку `react-i18next` для React.

Кожна текстова константа в інтерфейсі повинна бути перекладена. У налаштуваннях Django встановити `USE_I18N=True` і додати мовні файли локалізацій. Це гарантує зручність використання платформи різними категоріями користувачів згідно з їхніми мовними уподобаннями ¹⁶.

Висновок: документація та технічні рішення повинні бути реалізовані з огляду на високі стандарти безпеки, масштабованості та зручності користувацького інтерфейсу. Платформа поєднує сучасний стек (Django REST на бекенді, Vite+React на фронтенді, PostgreSQL як БД, Celery+Redis для асинхронних завдань) і має забезпечити гнучке створення чат-ботів для Telegram, Viber та WhatsApp з можливістю розширення в майбутньому.

Джерела: згадані рішення та підходи підтверджені в офіційній документації і публічних гайдах: Django-Allauth та DRF (автентифікація, throttling) ¹ ¹⁵, Celery з Redis для асинхронної обробки ⁸ ⁹, Stripe інтеграція через `dj-stripe` ⁶, контейнеризація Docker/Kubernetes ¹⁰ ¹¹, використання HTTPS і безпечного зберігання секретів ¹² ¹³, підтримка i18n у Django ¹⁶ тощо. Ці джерела підтверджують вибір технологій і практик, що наведені у специфікації.

¹ GitHub - pennersr/django-allauth: Integrated set of Django applications addressing authentication, registration, account management as well as 3rd party (social) account authentication. Mirror of <https://codeberg.org/allauth/django-allauth/>
<https://github.com/pennersr/django-allauth>

- 2 User Groups with Custom Permissions in Django - GeeksforGeeks
<https://www.geeksforgeeks.org/python/python-user-groups-custom-permissions-django/>
- 3 13 3 Steps to Build a Telegram Bot with Django Python | Towards Data Engineering
<https://medium.com/towards-data-engineering/3-steps-to-build-a-telegram-bot-with-django-python-a87e5cc1226c>
- 4 Viber REST API | Viber Developers Hub
<https://developers.viber.com/docs/api/rest-bot-api/>
- 5 How to Build an InvestorGPT AI Chatbot on WhatsApp with Django, Twilio, and the ChatGPT API | Twilio
<https://www.twilio.com/en-us/blog/developers/community/build-investor-ai-chatbot-whatsapp-django-chatgpt-api>
- 6 dj-stripe - Stripe Made Easy for Django Developers
<https://dj-stripe.dev/>
- 7 Adding Vite to Django for a Modern Front End with React and Tailwind CSS (Part of "Modern JavaScript for Django Developers") : r/django
https://www.reddit.com/r/django/comments/1p5g975/adding_vite_to_django_for_a_modern_front_end_with/
- 8 9 Asynchronous Tasks With Django and Celery – Real Python
<https://realpython.com/asynchronous-tasks-with-django-and-celery/>
- 10 11 Containerization and DevOps for Django: Docker, Kubernetes, and Beyond | by Yogeshkrishnanseeniraj | Medium
<https://medium.com/@yogeshkrishnanseeniraj/containerization-and-devops-for-django-docker-kubernetes-and-beyond-b1307b645296>
- 12 API Security in Django: Approaches, Trade-offs, and Best Practices - DEV Community
<https://dev.to/topunix/api-security-in-django-approaches-trade-offs-and-best-practices-nk4>
- 14 15 Throttling - Django REST framework
<https://www.django-rest-framework.org/api-guide/throttling/>
- 16 Internationalization and localization | Django documentation | Django
<https://docs.djangoproject.com/en/6.0/topics/i18n/>